

The AI helper, its memory, and its skills

What was built in Open Hospitality's desktop edition, why it was built that way, and what an owner gets from it. September 2026.

In one paragraph

Open Hospitality reads your night audit reports into real books without any AI at all. The AI helper is an optional extra you switch on, run on **your own AI account**, that does three things: it suggests where an unfamiliar charge code belongs, it reads a report from a front-desk system the app has no reader for, and it remembers what it learned so the next report costs nothing. It only ever *suggests* — nothing reaches your books until you press Confirm, in your name. It is never shown a guest's name, a pay rate, a bank detail or an account number, and that is enforced by how the questions are built, not by hoping.

Why an AI at all

Every front-desk system labels charges with short codes — RM, VI, T1, PET, MISC|PET_FEE. The app ships a dictionary of the common ones, but codes are set up per hotel, so some are guesses and some are unknown. An unknown code is the one that matters: its money stays out of your profit and loss until somebody says where it goes. That "somebody" used to be you, reading a list of codes and picking USALI lines.

The AI helper reads the same list and proposes a line with a reason — "PET_FEE: a pet charge is a room surcharge; Miscellaneous Income › Pet Fee" — so your job becomes agreeing or correcting, not researching. On tax questions it is told to refuse rather than guess, because a wrong tax line is worse than an unanswered one.

The five pieces

1. Your own account, your own limit

You choose who you have an account with — OpenRouter (one account for Claude, GPT, Gemini and more), Anthropic directly, OpenAI directly, a model running on your own computer (Ollama or LM Studio), or a practice mode that answers offline and costs nothing. The model is picked from a list with prices; nobody has to know that Claude on OpenRouter is spelled `anthropic/claude-sonnet-5`.

Your key goes into Windows' own password store under an entry for this install. It is never written to the database, a config file, a log, a backup or an error message. A backup restored on another computer asks for the key again.

There is a hard monthly limit, \$10 to begin with, and a count of questions beside it. The app stops *before* the call that would cross either and tells you which. Prices are never invented: if a model's price isn't known, the app counts questions instead of dollars and says so.

What you get: no subscription to us, no margin, no surprise bill, and a choice of model you can change any time.

2. Questions that are safe by construction

When the helper is asked about a code, the question is built from a fixed object with named fields: the front-desk system, the code, the description as printed on the report, the dates it was seen, the amounts, and the lines it could be filed under. There is no path from a database row to the prompt except through that object — so an employee's name, a pay rate, an SSN, a bank detail or an account number is *structurally absent*. The queries that build the question never touch a payroll, staff or guest table.

As a second line, the fully written request is scanned just before it leaves, and if anything forbidden is found the request is **refused, not masked**. A mask would hide a bug; a refusal makes it a visible error. Every question that leaves is recorded: what it was about, what it cost, and how it ended.

What you get: the same guarantee whichever model you pick, and a record you can audit.

3. Reading a report nobody has a reader for

Some front-desk systems the app cannot read on its own. For those, "Read with AI" shows the model the report — but a night audit pack is full of things that must never leave: guest names, account numbers, balances. So the report is split into pages, every page is run through the same scan, and only pages carrying nothing forbidden are sent, **whole**. Nothing is edited to make a page acceptable; a page goes entirely or not at all, and you are told how many were held back and why. On the real pack that prompted this, 21 of 48 pages survive and not one carries a person's name — and those 21 are the summary, which is the only part the books need.

What comes back is a proposal: rows with code, description and amount, and the business date. You look at them and press "Put these in the books", or you don't.

What you get: a hotel on an unusual system still keeps books, with the same privacy rule as everything else.

4. Skills — how each kind of report is read

A *skill* is knowledge about a **kind** of report: which page holds the figures, which column is the night's total, that a bracketed amount is money going out, where the business date prints. The app ships one reading guide per front-desk system as a Markdown file, versioned with the app's own readers, and gives the guide to the model when it reads a report. The model follows instructions instead of guessing the layout from scratch — fewer tokens, fewer misreads, the same rules the built-in readers follow. Guides carry no hotel data, so they are safe to show.

What you get: the model reads your report the way the app would, and you can open the guide and read the rules yourself.

5. Memory — what it has learned about *your* hotels

Memory is different from skill: it is what has been learned about **this owner's** hotels, and the owner is the only one who can change it.

- **Code decisions.** Every code you confirm — or an AI suggestion you accept — is recorded per hotel with who decided and when. The next report with that code is filed without a question.
- **Learned layouts.** After the model reads a report with no reader and you confirm the rows, the app works out a *recipe* that reproduces exactly those rows from the same pages. The model never writes the recipe; the app infers it from a fixed set of layouts and keeps it only if replaying it gives the confirmed rows exactly. From then on every report of that shape is read by replaying the recipe: **no model call, no cost, and the same reading every morning**. If a report's shape drifts, the recipe finds nothing, the read goes back to the helper, and you are told.
- **The database is the memory.** These facts live in your books. What is written to files is a mirror of them.

The mirror you can read: Obsidian

An Obsidian vault is just a folder of Markdown files, so the memory is written as notes into **Documents > Open Hospitality > AI memory**:

- **Start here** — a map of the vault.
- **One note per hotel** — its code, ownership entity, front-desk system, the reports read (which kinds, how many nights, first and latest), the layouts learned, and every charge code decided: what the report prints, where it is filed, who decided (you, or an AI suggestion you accepted) and when.
- **One reading guide per system** — the skill the model is given.

Open the folder in Obsidian (free; "Open folder as vault") to browse it with links between hotels, guides and layouts — or read the files in Notepad. Nobody has to install Obsidian for the app to work.

The notes are **a mirror, one way**. The app rewrites them from your books whenever they change, and only when they change. Editing a note does not alter how your reports are read; make changes in the app. A hand edit that silently changed how figures are filed is exactly the failure this design exists to prevent. The notes hold hotel names, codes and decisions — never a guest, an employee, an amount or a rate — so if the folder sits in a synced drive, that is all the sync carries.

What it never does

- It never writes to your books. A person applies every suggestion.
- It never sees a person: no guest names, staff names, pay rates, bank details or account numbers, by construction and by a scan that refuses.
- It never runs unasked. It is off until you switch the module on and supply an account; the only thing that reaches the internet is your own model.
- It never invents a price or a rule. Unknown prices count questions; tax questions are refused.
- It never learns on its own. Memory is written only from what you confirmed.

How an owner benefits, day to day

- **The first month:** instead of researching 10–20 codes, you read a list of suggestions with reasons and press "These look right — confirm all", changing the one or two you disagree with.
- **Every day after:** codes already decided are filed without a question; a report shape already learned is read with no model call. The helper is mostly silent, which is the point.
- **A hotel on an unusual system:** still keeps books, from the summary pages alone.
- **Trust:** you can open the AI memory folder and see exactly what the helper knows and why each code is filed where it is — in files, in plain words, with links.
- **Cost:** a fraction of a cent per question, a monthly ceiling you set, and zero for anything already learned.

Setting it up (two minutes)

1. **Modules** → turn on the AI helper.
2. **AI helper** → who you have an account with → pick the model from the list → paste your key → **Save**.
3. Press **Check it works**. It asks one fixed question that contains nothing about your hotel and shows what that cost.
4. Go to **Codes to confirm**. "Ask the AI" appears beside each code.

The Overview's *What's connected* card shows the helper's state: not set up, needs attention (a key missing, or never checked), or connected with the model and when it was last checked.

What is next

- A layout note per learned recipe in the vault, with the recipe as a readable table.
- Noticing a hand edit to a note and offering it back as a proposal to confirm in the app.
- More reading guides as more front-desk systems are met.

Source: src/usali/desktop/ai/ (allow-list, spend cap, adapters, page filter, recipes), src/usali/desktop/memory_notes.py (the vault), mapping/reading-guides/ (the skills), and the design record in docs/desktop/M4-ai-and-ledger.md.